UNIVERSIDAD DE MANIZALES

Facultad de Ciencias e Ingenierías

Programación I

Decisiones de Diseño del Sistema de Gestión Académica

Actividad colaborativa 2
Evolución del proyecto

Hernan Felipe Astudillo Peña

Entrega individual

Usuario de GitHub feliastu

31 de octubre de 2026

Repositorio
github.com/feliastu/programacion-i-gestion-academica

Introducción

La segunda entrega amplía el Sistema de Gestión Académica para ofrecer asignaturas en grupos y gestionar sus matrículas. La solución conserva las clases y operaciones de la primera actividad e incorpora abstracción, composición y excepciones personalizadas en el mismo proyecto. Este documento justifica esas decisiones y las relaciona con los métodos implementados.

Descripción general de la solución

El punto de entrada continúa siendo `main.py`. `crear_datos_demo()` construye los dos programas, las tres asignaturas y las seis personas utilizados en AC1. El programa mantiene las validaciones, la actualización conjunta del avance académico y el reporte por roles. Luego `demostrar_grupos()` utiliza esos objetos para crear dos grupos, matricular y retirar estudiantes, consultar información y controlar los errores solicitados.

Los modelos están en `modelos/`; las excepciones, en `excepciones.py`; y las pruebas, en `pruebas/`. `CatalogoAcademico` reúne referencias a programas y asignaturas para buscarlos por código. Esta clase permite resolver las búsquedas dentro del mismo sistema. Todos los datos de la demostración son ficticios y se mantienen en memoria.

Persona como clase abstracta

`Persona` concentra identificación, nombre y correo, junto con su inicialización y validación. Sin embargo, la actividad principal depende del rol académico. Por eso la clase hereda de `ABC` y declara `realizar_actividad_principal()` con `@abstractmethod`. Intentar construir directamente una Persona produce `TypeError`; una subclase que no implemente el método tampoco puede instanciarse.

`mostrar_informacion()` conserva una implementación común. Cada subclase la amplía mediante `super().mostrar_informacion()`, y su constructor reutiliza `super().__init__()`. Así, la abstracción exige el comportamiento particular sin duplicar los datos comunes.

Implementación en las clases derivadas

`Estudiante.realizar_actividad_principal()` describe el programa y semestre que cursa. `Docente` indica la facultad donde orienta clases. `Administrativo` presenta el cargo y la dependencia en que trabaja. Las tres implementaciones devuelven texto y conservan los comportamientos de AC1. El programa principal se ocupa de imprimirlo.

Relaciones entre los objetos

GrupoAcademico representa una oferta concreta de una asignatura. Su constructor recibe objetos ya creados de Asignatura y Docente, además del código del grupo, la capacidad y los datos del horario. Los estudiantes se incorporan mediante matrícula y también se conservan como objetos. Los getters permiten consultar sus datos actuales sin duplicar nombres o códigos como sustitutos de la relación.

Relación	Implementación y significado
Grupo y asignatura	__asignatura referencia la Asignatura ofrecida. Dos grupos pueden compartirla.
Grupo y docente	__docente referencia al Docente responsable, recibido en el constructor.
Grupo y estudiantes	__estudiantes contiene los objetos Estudiante matriculados.
Grupo y horario	__horario contiene una instancia de Horario creada por el grupo.
Asignatura y programa	Se conserva el objeto ProgramaAcademico asociado desde AC1.
Estudiante y programa	Se conserva el programa que cursa cada estudiante.

Composición del horario

El constructor del grupo ejecuta self.__horario = Horario(dia, hora_inicio, hora_fin, salon). Cada grupo crea así su propio horario, con día, inicio, finalización y salón encapsulados. La relación expresa que el grupo tiene una parte que describe cuándo y dónde se desarrolla. Heredar de Horario expresaría que el grupo es un horario y mezclaría la gestión de matrículas con los datos temporales.

El grupo controla la creación y el reemplazo de esa parte. cambiar_horario() construye primero un Horario válido; si la construcción falla, conserva el anterior. Horario ofrece métodos de consulta y no setters. Esta decisión muestra la composición mediante responsabilidad de creación y uso; no implica una garantía de destrucción inmediata en Python si otra variable conserva una referencia al horario.

Protección de la matrícula

get_estudiantes() devuelve una copia de la lista. Vaciar o ampliar esa copia no modifica las matrículas internas, aunque sus elementos siguen siendo los mismos objetos Estudiante. Los cupos disponibles se calculan como capacidad menos cantidad de matriculados. El grupo evita así mantener dos registros de ocupación que pudieran quedar desactualizados.

Matrícula y manejo de excepciones

matricular_estudiante(estudiante) comprueba que recibe un Estudiante, busca su código entre los matriculados y verifica el cupo disponible. Se comprueba primero el duplicado para informar esa causa incluso cuando el grupo está lleno. Solo después se agrega el objeto a la lista. retirar_estudiante(codigo_estudiantil) busca el código, elimina la referencia y devuelve el estudiante; si no existe, conserva la lista.

Las cinco excepciones se definen como clases propias en excepciones.py y heredan de ErrorAcademico, cuya base es Exception. Cada método lanza el error en el punto donde detecta el problema. El mensaje incorpora el código involucrado para que el resultado se entienda desde la consola.

Situaciones y puntos de generación

EstudianteYaMatriculadoError se lanza en GrupoAcademico.matricular_estudiante() cuando el código estudiantil ya figura en el grupo. La comparación por código también detecta otro objeto con el mismo código.

CapacidadMaximaError se genera en el mismo método cuando no quedan cupos. La operación termina antes de añadir al estudiante, por lo que no cambia la lista ni la ocupación.

EstudianteNoMatriculadoError se lanza en GrupoAcademico.retirar_estudiante() si el recorrido no encuentra el código solicitado. No se elimina ningún estudiante.

AsignaturaNoEncontradaError se genera en CatalogoAcademico.buscar_asignatura() cuando el código no está registrado. ProgramaNoEncontradoError se genera de manera equivalente en buscar_programa(). Las búsquedas válidas devuelven los objetos originales.

Captura y continuidad

demostrar_grupos(), en main.py, usa un bloque try/except específico para cada situación. Tras imprimir el error, continúa con una consulta o una nueva operación. La ejecución termina mostrando ambos grupos y un mensaje de finalización. Las clases del modelo no imprimen el error ni ocultan la excepción.

Los datos con formato o valor inválido mantienen ValueError, como en AC1. La creación directa de Persona produce el TypeError previsto por el mecanismo abstracto. Estos dos errores incorporados se distinguen de las cinco excepciones del dominio académico.

Polimorfismo y verificación

El reporte de AC1 permanece en `mostrar_reporte(personas)`. Recibe una colección heterogénea con dos estudiantes, dos docentes y dos administrativos. Un único ciclo llama a `mostrar_informacion()` y a `realizar_actividad_principal()` en cada objeto. La implementación concreta depende del objeto recibido y no de un condicional que pregunte por su tipo.

Convertir Persona en abstracta mantiene este funcionamiento: los tres roles ya implementan el método exigido. La prueba con una subclase Invitado, definida solo para verificar el reporte, confirma que puede incorporarse otro rol sin modificar el ciclo. Las comprobaciones de tipos en los constructores validan las asociaciones y no seleccionan las respuestas del reporte.

Demostración de la segunda entrega

PROG1-A y PROG1-B comparten el objeto de Programación I, tienen docentes diferentes y horarios independientes. El primer grupo tiene capacidad 1 y el segundo, capacidad 2. Se matricula a Ana en el primero y a Carlos en el segundo; luego se prueban el duplicado de Ana y la matrícula de Carlos en el grupo lleno.

El retiro de Ana libera un cupo. Se intenta retirar EST999, se controla el error y se matricula a Carlos en el lugar disponible. Después se buscan ASIG999 y PROG999; cada error se sigue de una búsqueda válida. Al terminar, el primer grupo tiene cero cupos libres y el segundo tiene uno. La pertenencia a un grupo no modifica la lista del otro.

Resultados de las pruebas

El 31 de octubre de 2026 se ejecutaron el programa y `python3 -m unittest discover -s pruebas -v` con Python 3.12.14. La demostración finalizó y las 42 pruebas pasaron: 16 del sistema previo, adaptadas a la abstracción, y 26 de la ampliación. Los archivos `docs/salida_demostracion.txt` y `docs/resultado_pruebas.txt` conservan las salidas reales.

Las pruebas comprueban abstracción, referencias compartidas, horarios independientes, matrícula, cupos, retiro y búsquedas. También revisan el estado después de los rechazos: no basta con producir un error si la operación deja cambios parciales. Un horario inválido conserva el anterior y una matrícula rechazada mantiene tanto la lista como los cupos.

Continuidad entre entregas

La versión anterior se conserva en el commit `aa1f190c328a5e4f4155d7db08570d21fbec4142`, identificado con Entrega_AC1. Los cambios de AC2 se registran mediante nuevos commits del mismo repositorio y se identifican con la etiqueta obligatoria Entrega_AC2. No se reemplaza el historial de la primera actividad.

Alcance y conclusiones

La solución integra los grupos y sus matrículas con las personas, programas y asignaturas ya existentes. Persona abstracta impide representar un rol incompleto; las subclases conservan la información y los comportamientos compartidos. La composición ubica la creación del horario en el grupo, mientras que las asociaciones reutilizan los objetos académicos disponibles.

Las excepciones separan la detección del problema en el modelo de su presentación en el programa principal. Validar antes de modificar evita matrículas duplicadas, capacidad excedida y retiros incorrectos. Las 42 pruebas respaldan estos comportamientos y la continuidad del reporte polimórfico.

La capacidad debe ser un entero positivo y el horario utiliza horas HH:MM dentro del mismo día, con final posterior al inicio. Los códigos se comparan exactamente tras quitar espacios externos. La comprobación de duplicados corresponde a cada grupo; no se añaden restricciones entre grupos, programas o facultades que no figuran en el enunciado.

La aplicación conserva datos en memoria y no implementa persistencia, cruces de horarios ni un registro global de códigos. Los identificadores de los objetos se mantienen estables durante la demostración. Estas limitaciones delimitan la versión entregada y permiten identificar futuras ampliaciones sin alterar los requisitos de AC2.

Declaración de uso de inteligencia artificial

Se utilizó ChatGPT de OpenAI para analizar el documento de la actividad, proponer e implementar la evolución del código, generar y ejecutar pruebas, y elaborar la documentación. También se conserva la declaración de asistencia utilizada en la primera entrega. El trabajo técnico se desarrolló a solicitud del estudiante responsable.

La asistencia no acredita por sí sola la comprensión del proyecto. La revisión, explicación y sustentación de las clases, métodos, pruebas y decisiones corresponden a Hernan Felipe Astudillo Peña, único estudiante responsable de esta entrega.

Referencias

Universidad de Manizales, Facultad de Ciencias e Ingenierías. Documento de la Actividad colaborativa N°2 sobre la evolución del Sistema de Gestión Académica, suministrado en el curso de Programación I.

Python Software Foundation. Documentación de Python 3.12. Consultada el 31 de octubre de 2026: clases base abstractas, errores y excepciones, y clases y referencias a objetos.

https://docs.python.org/es/3.12/library/abc.html

https://docs.python.org/es/3.12/tutorial/errors.html

https://docs.python.org/es/3.12/tutorial/classes.html